COMPUTACIÓN GRÁFICA · TERCERA PRÁCTICA CALIFICADA

Restauración y Super-Resolución de Video con SRGAN

Rstauracion de un video de la televisión peruana en los años 2000s.

302×240 SD a 1208×960 HD

José Manuel Montalvo Espinoza · Facultad de Ciencias — Universidad Nacional de Ingeniería

01 · PLANTEAMIENTO Y RECURSOS

Objetivo y Material a Usar

OBJETIVO GENERAL

Super-Resolución ×4 en Archivo SD

Restaurar secuencias degradadas (302×240, 185 kbps) a calidad HD (1208×960) suprimiendo bloques de compresión sin alterar la pista de audio original.

Dataset CelebA-HQ (Kaggle)
3,000 rostros HD para entrenamiento supervisado.

Secuencia de Prueba (2,134 cuadros)
Segmento de "El Especial del Humor" (TV Perú).

Entorno de Cómputo
TensorFlow/Keras, OpenCV, FFmpeg y GPU Colab.

02 · METODOLOGÍA DE RESTAURACIÓN

Flujo de Trabajo: Cómo se Mejora el Video

1. DEGRADACIÓN

Simular TV Analógica

Blur gaussiano, escala ×0.25 y JPEG agresivo.

→

2. ENTRENAMIENTO

Aprendizaje Híbrido

10 épocas MAE + 30 épocas GAN perceptual.

→

3. INFERENCIA

Reescalado ×4 GPU

Micro-lotes de 4 cuadros a 1208×960 px.

→

4. EXPORTACIÓN

Audio + Auditoría

Fusión FFmpeg (CRF 18) y comparativa visual.

02 · IMPLEMENTACIÓN DEL PIPELINE (ETAPAS 1 Y 3)

Código: Degradación Sintética e Inferencia GPU

1. Simulación de TV Analógica (CelebA-HQ)

```
img_blur = cv2.GaussianBlur(hr_crop, (5, 5), 1.1)
lr_img = cv2.resize(
    img_blur, (LR_SIZE, LR_SIZE),
    interpolation=cv2.INTER_AREA
)
quality = random.randint(30, 65)
encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), quality]
_, encimg = cv2.imencode('.jpg', lr_img, encode_param)
lr_img = cv2.imdecode(encimg, 1)
```

2. Inferencia por Micro-Lotes (BATCH = 4)

```
batch_tensor = np.array(
    batch_frames, dtype=np.float32
) / 255.0

sr_batch = generator.predict(batch_tensor, verbose=0)
sr_batch = np.clip(sr_batch * 255.0, 0,
255).astype(np.uint8)

for sr_f in sr_batch:
    out_sr.write(cv2.cvtColor(sr_f, cv2.COLOR_RGB2BGR))
```

Código: Audio con FFmpeg y Auditoría Zoom

3. Fusión de Video HD + Audio Original

```
subprocess.run([
    "ffmpeg", "-y",
    "-i", temp_sr_video, "-i", VIDEO_INPUT,
    "-c:v", "libx264", "-crf", "18",
    "-pix_fmt", "yuv420p",
    "-map", "0:v:0", "-map", "1:a:0?",
    "-c:a", "aac", "-shortest", FINAL_OUTPUT
], check=True)
```

4. Split-Screen hstack + Recorte t = 70s

```
subprocess.run([
    "ffmpeg", "-y", "-i", temp_bic, "-i", temp_sr,
    "-filter_complex", "hstack=inputs=2", COMP_OUT
], check=True)

cap.set(cv2.CAP_PROP_POS_FRAMES, int(70 * fps))
crop_sr = sr_frame[cy-125:cy+125, cx-125:cx+125]
```

Modelos Utilizados y su Rol

Modelo	Arquitectura	Configuración Clave	Rol en el Proyecto
Generador (G)	**SRResNet**	8 bloques residuales + 2 PixelShuffle	Limpia bloques y escala 302×240 a 1208×960.
Discriminador (D)	**CNN Clasificadora**	7 capas Conv2D + LeakyReLU(0.2)	Distingue fotogramas reales de sintéticos.
Extractor Perceptual	**VGG19 (ImageNet)**	Congelada en capa block5_conv4	Evalúa nitidez perceptual de texturas y rasgos.

Código: PixelShuffle y Función de Pérdida

Capa Sub-Píxel (×2 por cada bloque)

```
@tf.keras.utils.register_keras_serializable()
class PixelShuffle(layers.Layer):
    def __init__(self, scale, **kwargs):
        super().__init__(**kwargs)
        self.scale = scale

    def call(self, inputs):
        return tf.nn.depth_to_space(
            inputs, self.scale
  )
```

Pérdida Híbrida + Label Smoothing (0.9)

```
real_labels = tf.ones((batch_size, 1)) * 0.9
percep_loss = tf.reduce_mean(
    tf.square(vgg_real - vgg_fake)
)
adv_loss = bce(tf.ones_like(fake_pred), fake_pred)
pixel_loss = mae(hr, fake_hr)

g_loss = (0.02 * percep_loss
          + 3e-3 * adv_loss + 1e-2 * pixel_loss)
```

04 · EVALUACIÓN CUANTITATIVA Y VISUAL

Resultados y Comparación (2,134 Fotogramas)

26.69 dB
PSNR Promedio (Fidelidad)

0.8738
SSIM Promedio (Estructura)

1208×960
Resolución HD Final (×4)

Interpolación Bicúbica (Línea Base)

- Conserva macrobloques y ruido analógico.
- Suaviza bordes y difumina contornos.

SRGAN Propuesto (Nuestro Modelo)

- Elimina granulado y artefactos JPEG.
- Transiciones limpias sin parpadeo temporal.


El Especial del Humor
El Especial del Humor
Detalle Zoom - Original
Detalle Zoom - SRGAN

05 · SÍNTESIS Y TRABAJO FUTURO

Conclusiones del Proyecto

Limpieza Visual (Denoising)

Supresión efectiva de bloques JPEG y ruido analógico.

Estabilidad Perceptual

Pérdida VGG19 preserva colores sin deformaciones.

Sesgo de Dominio (CelebA-HQ)

Mejora sutil en texto y planos generales dinámicos.

Trabajo Futuro

Entrenar con DIV2K y consistencia temporal.



ENLACES Y MAS DETALLES DEL PROYECTO:

https://colab.research.google.com/drive/1eYZUv6bfUit3JpE6-wtVS7qzSK_ByYij?usp=sharing

https://github.com/manuel270599/PC3_ComputacionGrafica

Muchas Gracias

Créditos del material: "El Especial del Humor"

dataset CelebA-HQ (Kaggle).